Program 5-26 *(continued)*

```
        // Display the total.
        cout << fixed << showpoint << setprecision(2);
        cout << "The total is $" << total << endl;
        return 0;
}
```

Program Output with Example Input Shown in Bold

```
How many DVDs are being rented? 6 [Enter]
Is DVD #1 a current release? (Y/N) y [Enter]
Is DVD #2 a current release? (Y/N) n [Enter]
DVD #3 is free!
Is DVD #4 a current release? (Y/N) n [Enter]
Is DVD #5 a current release? (Y/N) y [Enter]
DVD #6 is free!
The total is $12.00
```

Case Study: See the Loan Amortization Case Study on the Computer Science Portal at www.pearsonglobaleditions.com/gaddis.

Review Questions and Exercises

Short Answer

1. Why should you indent the statements in the body of a loop?
2. Describe the difference between pretest loops and posttest loops.
3. Why are the statements in the body of a loop called conditionally executed statements?
4. What is the difference between the `while` loop and the `do-while` loop?
5. Which loop should you use in situations where you wish the loop to repeat until the test expression is false, and the loop should not execute if the test expression is false to begin with?
6. Which loop should you use in situations where you wish the loop to repeat until the test expression is false, but the loop should execute at least one time?
7. Explain the concept of a user-controlled loop. Which loop(s) can be used as a user-controlled loop? Give an example to support your answer.
8. Why is it critical that counter variables be properly initialized?
9. Why is it critical that accumulator variables be properly initialized?
10. Why should you be careful not to place a statement in the body of a `for` loop that changes the value of the loop's counter variable?
11. What header file do you need to include in a program that performs file operations?
12. What data type do you use when you want to create a file stream object that can write data to a file?
13. What data type do you use when you want to create a file stream object that can read data from a file?
14. Why should a program close a file when it's finished using it?

15. What is a file's read position? Where is the read position when a file is first opened for reading?

Fill-in-the-Blank

16. To __________ a value means to increase it by one, and to __________ a value means to decrease it by one.
17. In the __________ mode of increment operator, the precedence of the increment operator is higher than all other operators in the expression except brackets.
18 When the increment or decrement operator is placed after the operand (or to the operand's right), the operator is being used in __________ mode.
19. The three looping structures that C++ supports are __________, __________, and __________.
20. The first line of a `while` loop or a `for` loop is generally called a __________.
21. A loop that evaluates its test expression before each repetition is a(n) __________ loop.
22. A __________ loop is a type of posttest loop.
23. A loop that does not have a way of stopping is a(n) __________ loop.
24. A block of statements is enclosed within __________.
25. A(n) __________ is a sum of numbers that accumulates with each iteration of a loop.
26. A(n) __________ is a variable that is initialized to some starting value, usually zero, then has numbers added to it in each iteration of a loop.
27. A(n) __________ is a special value that marks the end of a series of values.
28. The __________ loop always iterates at least once.
29. The __________ and __________ loops will not iterate at all if their test expressions are false to start with.
30. The __________ loop is ideal for situations that require a counter.
31. Inside the `for` loop's parentheses, the first expression is the __________, the second expression is the __________, and the third expression is the __________.
32. A loop that is inside another is called a(n) __________ loop.
33. The header file `fstream` contains all declarations necessary for __________.
34. An object of __________ data type is created when an existing file is opened such that the data can be read from it.

Algorithm Workbench

35. Write a `while` loop that asks the user for a number (integer) and finds the sum of all the numbers entered. The process is continued till the sum of the numbers exceeds 100. Initialize the sum as 0 and display all partial sums.
36. Write a `do-while` loop that asks the user for a number. The square root of the number is then computed and displayed. The process continues as long as the user inputs a positive number; otherwise the loop terminates. Assume that the first number is 1.
37. Write a `for` loop to display the following set of numbers:

    `2, 4, 8, 16 ... 1024, 2048`
38. Write a `for` loop to display all the even numbers from 100 to 250. Also include a counter variable that counts the number of even numbers generated.
39. Write a nested loop that displays 10 rows of '#' characters. There should be 15 '#' characters in each row.

40. Convert the following `while` loop to a `do-while` loop:

```
int x = 1;
while (x > 0)
{
    cout << "enter a number: ";
    cin >> x;
}
```

41. Convert the following `do-while` loop to a `for` loop:

```
int response;
do
{
    cout << "Hello! Press 1 to exit loop...";
    cin >> response;
} while ( response != 1 );
```

42. Convert the following `while` loop to a `for` loop:

```
int count = 0;
while (count < 50)
{
    cout << "count is " << count << endl;
    count++;
}
```

43. Convert the following `for` loop to a `while` loop:

```
for (int x = 50; x > 0; x--)
{
    cout << x << " seconds to go.\n";
}
```

44. Write code that does the following: Opens an output file with the filename Numbers.txt, uses a loop to write the numbers 1 through 100 to the file, then closes the file.
45. Write code that does the following: Opens the Numbers.txt file that was created by the code you wrote in Question 44, reads all of the numbers from the file and displays them, then closes the file.
46. Modify the code that you wrote in Question 45 so it adds all of the numbers read from the file and displays their total.

True or False

47. T F The operand of the increment and decrement operators can be any valid mathematical expression.
48. T F The value of the variable `z` after execution of the following statements will be 11:

```
x = y = 5;
z = x + ++y;
```

49. T F The value of the variable `z` after execution of the following statements will be 11:

```
x = y = 5;
z = x++ + y;
```

50. T F The `while` loop is a pretest loop.
51. T F The `do-while` loop is a pretest loop.
52. T F The `for` loop is a posttest loop.
53. T F It is not necessary to initialize counter variables.

54. T F Omission of test expression in a `for` loop results in an infinite loop.
55. T F In a `for` loop, multiple test expressions may be present and each test expression is separated from the next by a comma.
56. T F Variables may be defined inside the body of a loop.
57. T F Variables may be defined in the conditional expression of a `while` loop.
58. T F In a nested loop, the outer loop executes faster than the inner loop.
59. T F In a nested loop, the inner loop goes through all of its iterations for every single iteration of the outer loop.
60. T F In a nested loop, if the outer loop has `m` iterations and the inner loop has `n` iterations, then the total number of iterations will be `m×n`.
61. T F The `break` statement causes a loop to stop the current iteration and begin the next one.
62. T F The `continue` statement causes a terminated loop to resume.
63. T F The stream extraction operator can be used to write data to a file.
64. T F When you call an `ofstream` object's `open` member function, the specified file will be erased if it already exists.

Find the Errors

Each of the following programs has errors. Find as many as you can.

65.
```
// Find the error in this program.
#include <iostream>
using namespace std;

int main()
{
    int num1 = 0, num2 = 10, result;

    num1++;
    result = ++(num1 + num2);
    cout << num1 << " " << num2 << " " << result;
    return 0;
}
```

66.
```
// This program adds and finds the greater of two numbers.
#include <iostream>
using namespace std;

int main()
{
    int num1, num2;
    char again = 'Y';
    do
    {
        cout << "Enter two numbers: ";
        cin >> num1 >> num2;
        if ( num1 > num2 ) cout << num1;
        cout << "Do you want to do this again? ";
        cin >> again;
    } while (again = 'y' || again = 'Y');
    return 0;
}
```

67.
```
// This program uses a loop to raise a number to a power.
#include <iostream>
using namespace std;

int main()
{
    int num, bigNum, power, count;

    cout << "Enter an integer: ";
    cin >> num;
    cout << "What power do you want it raised to? ";
    cin >> power;
    bigNum = num;
    while (count++ < power);
        bigNum *= num;
    cout << "The result is << bigNum << endl;
    return 0;
}
```

68.
```
// This program averages a set of numbers.
#include <iostream>
using namespace std;

int main()
{
    int numCount, total;
    double average;

    cout << "How many numbers do you want to average? ";
    cin >> numCount;
    for (int count = 0; count < numCount; count++)
    {
        int num;
        cout << "Enter a number: ";
        cin >> num;
        total += num;
        count++;
    }
    average = total / numCount;
    cout << "The average is << average << endl;
    return 0;
}
```

69.
```
// This program displays the sum of two numbers.
#include <iostream>
using namespace std;

int main()
{
    int choice, num1, num2;
```

```
    do
    {
        cout << "Enter a number: ";
        cin >> num1;
        cout << "Enter another number: ";
        cin >> num2;
        cout << "Their sum is " << (num1 + num2) << endl;
        cout << "Do you want to do this again?\n";
        cout << "1 = yes, 0 = no\n";
        cin >> choice;
     } while (choice = 1)
     return 0;
}
```

70.
```
// This program displays the sum of the numbers 1-100.
#include <iostream>
using namespace std;

int main()
{
    int count = 1, total;

    while (count <= 100)
        total += count;
    cout << "The sum of the numbers 1-100 is ";
    cout << total << endl;
    return 0;
}
```

Programming Challenges

1. Sum of Numbers

Write a program that asks the user for a positive integer value. The program should use a loop to get the sum of all the integers from 1 up to the number entered. For example, if the user enters 50, the loop will find the sum of 1, 2, 3, 4, . . ., 50.

Input Validation: Do not accept a negative starting number.

2. Multiplication Tables

Write a program that lets the user enter a number and then displays the multiplication table till 10 for that number.

3. Conversion Table from Degrees to Radians

Write a program to generate a table of conversion from degrees to radians. The value of degrees should start with 0° and end with 90°. Increment each row from the previous one by 10°. (Note that $\pi = 180° = 3.141593$ radians.)

VideoNote

Solving the Calories Burned Problem

4. Calories Burned

Running on a particular treadmill you burn 3.6 calories per minute. Write a program that uses a loop to display the number of calories burned after 5, 10, 15, 20, 25, and 30 minutes.

5. Harmonic Sum

Write a program that calculates the nth harmonic number H_n using the following formula:

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

6. **Distance Traveled**

The distance a vehicle travels can be calculated as follows:

```
distance = speed * time
```

For example, if a train travels 40 miles per hour for 3 hours, the distance traveled is 120 miles.

Write a program that asks the user for the speed of a vehicle (in miles per hour) and how many hours it has traveled. The program should then use a loop to display the distance the vehicle has traveled for each hour of that time period. Here is an example of the output:

```
What is the speed of the vehicle in mph? 40
How many hours has it traveled? 3
Hour  Distance Traveled
--------------------------------
 1           40
 2           80
 3          120
```

Input Validation: Do not accept a negative number for speed and do not accept any value less than 1 for time traveled.

7. **Arithmetic Series**

Write a program that asks the user for three values: an initial value for an arithmetic series, a second value for the series, and the number of values that should be calculated and displayed. The difference between the second and the first value determines the increment for the series.

Input Validation: The number of values has to be greater than 0.

Alternate Version: Write a program that takes as the third value an upper limit for the series. If the user enters 100 as the third value, the series stops as soon as it equals or exceeds 100.

8. **Math Tutor**

This program started in Programming Challenge 17, of Chapter 3, and was modified in Programming Challenge 11 of Chapter 4. Modify the program again so it displays a menu allowing the user to select an addition, subtraction, multiplication, or division problem. The final selection on the menu should let the user quit the program. After the user has finished the math problem, the program should display the menu again. This process is repeated until the user chooses to quit the program.

Input Validation: If the user selects an item not on the menu, display an error message and display the menu again.

9. **The Collatz Conjecture**

Write a program that lets the user enter a positive integer and then reduces that value to 1 by performing the following steps:

- If the value of n is even, continue with $n/2$.
- If n is odd, continue with $3n + 1$.

Display the intermediate steps and finally display the number of steps needed to reach 1.

Input Validation: Make sure that the number entered is greater than 0.

The conjecture that this always converges to 1 was formulated by Lothar Collatz in 1937 and is still an unsolved problem in mathematics, despite a claim of proof in 2011.

10. Palindrome Check

A number is called a palindrome if it remains unchanged when its digits are reversed. Write a program to accept a nonzero positive number. The program should loop till the user inputs a valid number. The program should then generate the reversed number. If the reversed number is equal to the original, a message should be displayed that the number is a palindrome.

Hint: To find the reverse of a number, you can follow this procedure to reverse a number n. Initialize reversed number revno to 0. Divide n by 10 and store the remainder in r. Then add the remainder r to revno multiplied by 10. Repeat continually until n becomes 0.

11. Population

Write a program that will predict the size of a population of organisms. The program should ask the user for the starting number of organisms, their average daily population increase (as a percentage), and the number of days they will multiply. A loop should display the size of the population for each day.

Input Validation: Do not accept a number less than 2 for the starting size of the population. Do not accept a negative number for average daily population increase. Do not accept a number less than 1 for the number of days they will multiply.

12. Celsius to Fahrenheit Table

In Programming Challenge 12 of Chapter 3, you were asked to write a program that converts a Celsius temperature to Fahrenheit. Modify that program so that it uses a loop to display a table of the Celsius temperatures 0–20, and the Fahrenheit equivalents.

13. Sum of a Series

Write a program that displays the following series and also finds its sum:

$$\frac{2}{1^2} + \frac{4}{2^2} + \frac{6}{3^2} + \frac{8}{4^2} + \frac{10}{5^2} + \cdots\cdots\cdots\cdots + \frac{20}{10^2}$$

14. Student Line Up

A teacher has asked all her students to line up according to their first name. For example, in one class Amy will be at the front of the line, and Yolanda will be at the end. Write a program that prompts the user to enter the number of students in the class, then loops to read that many names. Once all the names have been read, it reports which student would be at the front of the line and which one would be at the end of the line. You may assume that no two students have the same name.

Input Validation: Do not accept a number less than 1 or greater than 25 for the number of students.

15. Estimated Wait Time

Waiting in queues is rarely enjoyable. What makes it worse is when you seem to always pick the slowest queue. Write a program that lets the user input the number of articles in each shopping cart in a queue. Each number entered corresponds to one shopping cart. To end the input, the user enters 0. The program should then display the estimated waiting time in minutes and seconds. The waiting time is composed of 0.75 seconds per article and an additional 30 seconds per shopping cart.

Input Validation: All values must be either strictly positive or 0.

16. Savings Account Balance

Write a program that calculates the balance of a savings account at the end of a period of time. It should ask the user for the annual interest rate, the starting balance, and the number of months that have passed since the account was established. A loop should then iterate once for every month, performing the following:

A) Ask the user for the amount deposited into the account during the month. (Do not accept negative numbers.) This amount should be added to the balance.
B) Ask the user for the amount withdrawn from the account during the month. (Do not accept negative numbers.) This amount should be subtracted from the balance.
C) Calculate the monthly interest. The monthly interest rate is the annual interest rate divided by 12. Multiply the monthly interest rate by the balance, and add the result to the balance.

After the last iteration, the program should display the ending balance, the total amount of deposits, the total amount of withdrawals, and the total interest earned.

> **NOTE:** If a negative balance is calculated at any point, a message should be displayed indicating the account has been closed and the loop should terminate.

17. Pattern Generator

Write a program the lets the user input in the following order:

- A character
- An integer `'n'` for the number of times this character has to be repeated
- Another character
- Another integer `'m'` for the number of times the second character has to be repeated
- Two integers for the number of rows and columns

The program should generate a pattern based on the input parameters. The pattern is composed of the repetition of `'n'` times the first character and `'m'` times the second character up until all rows have been filled with the specified number of characters.

If, for example, '*' must be repeated thrice and '+' twice in 4 rows of 21 columns, the pattern looks like this:

```
***++***++***++***++*
**++***++***++***++**
*++***++***++***++***
++***++***++***++***+
```

18. Population Bar Chart

Write a program that produces a bar chart showing the population growth of Prairieville, a small town in the Midwest, at 20-year intervals during the past 100 years. The program should read in the population figures (rounded to the nearest 1,000 people) for 1900, 1920, 1940, 1960, 1980, and 2000 from a file. For each year, it should display the date and a bar consisting of one asterisk for each 1,000 people. The data can be found in the `People.txt` file.

Here is an example of how the chart might begin:

```
PRAIRIEVILLE POPULATION GROWTH
(each * represents 1,000 people)
1900 **
1920 ****
1940 *****
```

19. Count Positive and Negative Numbers

Write a program that asks the user for a positive integer n within 10 to 20. The program should loop till the user inputs a valid number. After receiving a valid input, it should prompt the user to enter n numbers within a loop and count the total number of positive and negative numbers separately.

20. Random Number Guessing Game

Write a program that generates a random number and asks the user to guess what the number is. If the user's guess is higher than the random number, the program should display "Too high, try again." If the user's guess is lower than the random number, the program should display "Too low, try again." The program should use a loop that repeats until the user correctly guesses the random number.

21. Random Number Guessing Game Enhancement

Enhance the program that you wrote for Programming Challenge 20 so it keeps a count of the number of guesses the user makes. When the user correctly guesses the random number, the program should display the number of guesses.

22. Square Display

Write a program that asks the user for a positive integer no greater than 15. The program should then display a square on the screen using the character 'X'. The number entered by the user will be the length of each side of the square. For example, if the user enters 5, the program should display the following:

```
XXXXX
XXXXX
XXXXX
XXXXX
XXXXX
```

If the user enters 8, the program should display the following:

```
XXXXXXXX
XXXXXXXX
XXXXXXXX
XXXXXXXX
XXXXXXXX
XXXXXXXX
XXXXXXXX
XXXXXXXX
```

23. Pattern Displays

Write a program that uses a loop to display Pattern A below, followed by another loop that displays Pattern B.

Pattern A	Pattern B
+	++++++++++
++	+++++++++
+++	++++++++
++++	+++++++
+++++	++++++
++++++	+++++
+++++++	++++
++++++++	+++
+++++++++	++
++++++++++	+

24. Using Files—Numeric Processing

If you have downloaded this book's source code from the Computer Science Portal, you will find a file named Random.txt in the Chapter 05 folder. (The Portal can be found at www.pearsonglobaleditions.com/gaddis.) This file contains a long list of random numbers. Copy the file to your system, then write a program that opens the file, reads all the numbers from the file, and calculates the following:

A) The number of numbers in the file
B) The sum of all the numbers in the file (a running total)
C) The average of all the numbers in the file

The program should display the number of numbers found in the file, the sum of the numbers, and the average of the numbers.

25. Using Files—Student Line Up

Modify the Student Line Up program described in Programming Challenge 14 so it gets the names from a file. Names should be read in until there is no more data to read. If you have downloaded this book's source code you will find a file named LineUp.txt in the Chapter 05 folder. You can use this file to test the program.

26. Personal Web Page Generator

Write a program that asks the user for his or her name, then asks the user to enter a sentence that describes himself or herself. Here is an example of the program's screen:

```
Enter your name: Julie Taylor [Enter]
Describe yourself: I am a computer science major, a member of the Jazz club,
and I hope to work as a mobile app developer after I graduate. [Enter]
```

Once the user has entered the requested input, the program should create an HTML file, containing the input, for a simple webpage. Here is an example of the HTML content, using the sample input previously shown:

```
<html>
<head>
</head>
<body>
   <center>
      <h1>Julie Taylor</h1>
   </center>
   <hr />
   I am a computer science major, a member of the Jazz club,
   and I hope to work as a mobile app developer after I graduate.
   <hr />
</body>
</html>
```

27. Average Steps Taken

A Personal Fitness Tracker is a wearable device that tracks your physical activity, calories burned, heart rate, sleeping patterns, and so on. One common physical activity that most of these devices track is the number of steps you take each day.

If you have downloaded this book's source code, you will find a file named steps.txt in the Chapter 05 folder. The steps.txt file contains the number of steps a person has taken each day for a year. There are 365 lines in the file, and each line contains the number of steps taken during a day. (The first line is the number of steps taken on January 1, the second line is the number of steps taken on January 2, and so forth.) Write a program that reads the file, then displays the average number of steps taken for each month. (The data is from a year that was not a leap year, so February has 28 days.)